

AutoLFD: A Three-Stage Framework for Microservice Fine-grained Auto-scaling in Edge Server Cluster

Kai Peng[✉], Zhanhao Zhang[✉], Shangwen Zhuang, Jiacheng Wang, Menglan Hu[✉],
Tianyue Zheng, and Zehui Xiong

Abstract—The advancement of edge computing and lightweight containerization technologies has spurred the adoption of the microservices architecture. However, microservice scheduling in edge environments faces significant challenges. The first challenge lies in the tight coupling between microservice deployment and request routing, which is often overlooked in existing research. This neglect will lead to the failure to obtain a satisfactory solution. Secondly, user requests in edge environments are highly unstable and difficult to predict. Existing static or passive scaling strategies cannot cope with this dynamic nature, resulting in the increase of user request latency and the decrease of system resource utilization. Finally, scheduling large-scale microservices usually incurs high computational overhead, and existing solutions that rely on redeployment are less cost-effective. At the same time, such methods are also unable to support fast, real-time adjustment, further limiting their practicality in real-world edge environments. Therefore, this paper proposes a new three-stage AutoLFD algorithm to address these challenges. This algorithm combines Lyapunov optimization, first-fit diminishing (FFD), and deep reinforcement learning (DRL) to perform fine-grained auto-scaling of microservices, minimizing user request latency while keeping system costs within a controllable range. Through detailed model analysis and strategy design, it jointly optimizes microservice deployment and request routing to ensure that the system obtains a satisfactory solution. Extensive experimental results demonstrate that AutoLFD significantly outperforms state-of-the-art algorithms, achieving substantial reductions in latency and cost.

Index Terms—Microservice auto-scaling, Edge computing, Lyapunov optimization, Deep reinforcement learning.

I. INTRODUCTION

WITH the rapid proliferation of the Internet of Things (IoT), and latency-sensitive AI applications [1], traditional cloud-centric architectures are increasingly inadequate to support the stringent demands of emerging services [2]. To bridge this gap, edge computing has emerged as a promising paradigm that brings computation and intelligence closer to

data sources [3], [4]. Edge server clusters are widely deployed in urban or industrial environments to provide high-performance and low-latency support for delay-sensitive tasks [5], [6]. Due to the limited resources of the edge environment, the traditional monolithic service architecture is no longer applicable. The microservice architecture has gained significant traction in edge computing scenarios due to its modular and distributed nature [7].

By decomposing monolithic applications into loosely coupled, independently deployable services, microservices promote scalability, fault isolation, and fine-grained resource management across heterogeneous edge nodes [8], [9]. However, despite the enormous resource management and scheduling advantages brought about by microservice architecture, orchestrating and scaling microservices in the cluster environment of edge servers remains a difficult challenge [10], [11]. The highly dynamic and resource-constrained edge environment, coupled with complex service dependencies and fluctuating user demands, imposes significant barriers to achieving efficient, low-latency, and energy-aware microservice management [12], [13]. These challenges call for adaptive and intelligent orchestration mechanisms to ensure service-level objectives (SLOs) under strict latency and resource constraints. However, there are still some limitations in the existing studies [14], [15].

First, due to the limited resources of edge servers, a complete service is hard to execute solely on a single server. Consequently, a service chain often needs to be decomposed into multiple microservices, which are then instantiated across different servers. This results in a complex optimization problem involving both microservice deployment and request routing. Moreover, there exists a strong coupling between deployment and routing, the two processes significantly influence each other. However, most existing studies treat microservice deployment and request routing as independent optimization tasks [16]–[18], neglecting their mutual interdependencies. Optimizing only microservice deployment or request routing often leads to optimal results in one dimension but creates bottlenecks or inefficiencies in the other dimension. Ultimately, preventing the global optimal solution of system performance.

Second, user requests in edge environments are dynamic and difficult to predict. Designing effective strategies to adapt to such fluctuating demands under constrained edge resources is a major challenge. Many existing studies optimize microservice deployment and routing based on the assumption that the request flow remains static [17], [19]. These static strategies cannot accommodate dynamic traffic patterns, often resulting

Manuscript received April 19, 2021; revised August 16, 2021.

This work is supported in part by the National Natural Science Foundation of China under Grant 62171189, and in part by the Key Research and Development Program of Hubei Province, China under Grants 2024BAA011, 2024BAB016, and 2024BAB031. (Corresponding author: Menglan Hu.)

Kai Peng, Zhanhao Zhang, Shangwen Zhuang, Jiacheng Wang, and Menglan Hu are with the Hubei Key Laboratory of Internet of Intelligence, School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: pkhust@hust.edu.cn; d202481226@hust.edu.cn; z15078382658@163.com; w651376825@gmail.com; humenglan@hust.edu.cn.)

Tianyue Zheng is with the School of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China (e-mail: zhengty@sustech.edu.cn)

Zehui Xiong is with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast (e-mail: z.xiong@qub.ac.uk)

in increased request failures and higher service latency under long-term traffic fluctuations. Some studies have considered the dynamic nature of request flows [20]. However, most adopt reactive adjustments that are made only after the traffic variation occurs. Such reactive strategies are inherently delayed and can be particularly problematic in handling chained microservice requests, where the blockage of a single microservice may lead to system-wide failure.

Finally, balancing operational cost with service quality is paramount for practical edge service deployment. Service providers must ensure quality of service (QoS) while maintaining economic viability through efficient resource utilization. Given the scarcity of edge resources, it is critical to reduce the number of microservice instances to avoid unnecessary consumption. Some studies propose re-deployment strategies to address dynamic request patterns [21], [22]. However, frequently shutting down and restarting a large number of microservice containers results in significant energy consumption and high operational costs. Additionally, re-deployment often incur long scaling times, making the system susceptible to instability under rapid traffic changes.

Aiming at the above research gaps, this paper aims to design a better auto-scaling method. However, The following challenges will be faced in the design process. First, due to the coupling relationship between microservice deployment and request routing, it is necessary to reasonably model the system, decouple them and implement joint optimization to ensure the optimal solution. Secondly, user requests in edge environments are complex and difficult to predict. At the same time, it is important to ensure the stability of the system. Therefore, how to accurately predict the request arrival rate of the next time slot and make adjustments in time is a great challenge. Finally, due to the limited cooling and power resources in the edge scenario and the strict control of overhead required by most private server clusters, it is also a challenge to guarantee low latency levels and QoS at a limited cost.

To this end, in order to address the above challenges and fill the research gap. This paper proposes an new three-stage algorithm called AutoLFD. This algorithm combines Lyapunov optimization, first-fit diminishing (FFD), and deep reinforcement learning (DRL) to perform fine-grained auto-scaling of microservices, minimizing user request latency while keeping system costs within a controllable range. Through detailed model analysis and strategy design, it jointly optimizes microservice deployment and request routing to ensure that the system obtains the near-optimal solution. In summary, the main contributions of this paper are as follows:

- This paper introduces the Jackson queuing theory and models the process of user requests waiting for microservice instance processing as a queuing system [23]. The entire microservice system is then modeled using an open Jackson queuing network, integrating queuing latency, communication latency, and processing latency to establish a comprehensive analytical model for system request-response latency.
- This paper proposes a three-stage algorithm, AutoLFD, to solve this NP-hard problem. First, based on Lyapunov optimization theory [24], we transform the long-term

stability problem of microservice deployment cost and request latency into an optimization problem within a single time slot. Then, based on the FFD method [25], the MFFD algorithm was designed for the initial deployment of the microservice system, which was regarded as a static deployment process. Finally, based on the results of Lyapunov optimization and DRL, MDRL algorithm was designed for auto-scaling of microservices.

The organization of this paper is as follows: In Section I, we introduce the background and contributions of this study. Section II presents the related work. In Section III, we model the network and formulate the problem. Section IV provides a detailed description of our proposed algorithm. Section V comprehensively presents the experimental results. Finally, Section VI concludes the paper.

II. RELATED WORK

A. Service Deployment

Research on microservice deployment focuses on efficiently placing loosely coupled but interdependent services on appropriate nodes to ensure system stability and operational efficiency. Wang et al. [26] studied resource allocation in cloud-edge collaborative environments under SLOs constraints, proposing a linear programming-based approximation algorithm to determine the optimal number of instances per server. Yang et al. [27] modeled the service function chain (SFC) deployment problem as a mixed integer nonlinear programming (MINLP) formulation and applied reinforcement learning for latency-sensitive scheduling. Li et al. [28] developed MOTAS, a scheduling framework that considers both microservice dependencies and cluster relationships, using a recursive graph mapping heuristic for cloud deployment. To enhance scalability, Liu et al. [29] proposed a multiobjective evolutionary algorithm for heterogeneous server clusters. Lv et al. [30] introduced GRLD, a graph reinforcement learning-based framework that reduces overhead while satisfying QoS constraints. However, these studies often ignore the impact of deployment strategies on request routing, and thus fail to achieve good results.

B. Request Routing

Microservices are often deployed across distributed environments with complex dependencies, leading to multiple routing options with varying latencies. The key challenge lies in minimizing average response time while balancing loads. Bhattacharya et al. [31] proposed BLOC, which balances load by analyzing system resource usage. Bachar et al. [32] designed MCOSS, adjusting load balancing weights using multi-cluster demand information. Wang et al. [33] formulated routing as a many-to-one matching game and proposed DDA-DQ, a dependency-aware algorithm with dynamic quotas. Zhao et al. [34] introduced a redundancy placement framework using sample average approximation to enhance responsiveness. However, most approaches ignoring its dependency on deployment decisions, which limits their effectiveness in high-concurrency scenarios.

TABLE I
COMMON SYMBOLS AND VARIABLES

Notation	Description
$\mathcal{S}$	Collection of data center servers
$\mathcal{M}$	Collection of microservices
$\mathcal{R}$	Collection of user request streams
T_{s_a, s_b}^{prop}	The inherent propagation latency between servers
B_{s_a, s_b}	The bandwidth between servers
T_r^{max}	Request response time limit
CR_s	CPU resources owned by server s
MR_s	Memory resources owned by server s
cr_m	CPU resources required to deploy the microservice m
mr_m	Memory resources required to deploy the microservice m
μ_m	Request processing rate of microservice m instances
$T_G(\cdot)$	Average request latency for a microservice graph
$c(\cdot)$	The total cost function
$\bar{C}$	Average cost estimates given by the service provider
N_m	The number of microservices
N_s	The number of servers
$N_{mchange}$	The number of changed microservice instances
$N_{schange}$	The number of changed servers
$L(\cdot)$	Lyapunov function
$Q(\cdot)$	Lyapunov virtual queue
$\Delta(\cdot)$	Lyapunov drift function
$D(\cdot)$	The elastic resource scheduling scheme

C. Joint Service Deployment and Request Routing

Deployment and routing are tightly interdependent placement decisions affect routing feasibility, while routing impacts instance loads. To address this, recent work explores joint optimization. Ren et al. [35] proposed JSORD, a mixed integer linear programming-based orchestration and scheduling mechanism. Mao et al. [36] presented a traffic-aware framework combining dynamic segmentation and nearest-neighbor routing. Hu et al. [37] designed a dual-layer reinforcement learning algorithm to achieve the joint optimization of microservice deployment and request routing. Peng et al. [38] proposed a 2-approximation algorithm using rounding for deployment and routing in mobile edge environments. Nevertheless, most methods rely on simplified queueing models like $M/M/1$ and assume static loads, limiting their adaptability to real-world, dynamic conditions.

D. Auto-scaling under Dynamic Load

To maintain performance under fluctuating workloads, microservice systems require timely resource scaling or migration. Some studies focus on instance migration. Zeng et al. [39] proposed ADVMC, a VM consolidation framework using deep reinforcement learning and host overload evaluation to reduce energy consumption. Liu et al. [40] presented E2MS, a migration strategy for dynamic manufacturing environments. However, these works often neglect broader scheduling constraints. Others target elastic scaling. Kardani et al. [41] introduced ADRL, which integrates anomaly detection and reinforcement learning for adaptive scaling. Rossi et al. [42] developed a scaling framework using dynamic thresholds, while Cheng et al. [43] proposed ProScale, leveraging workload prediction for cloud-edge collaboration. Still, most methods treat scaling and migration separately, lacking unified strategies for dynamic load balancing.

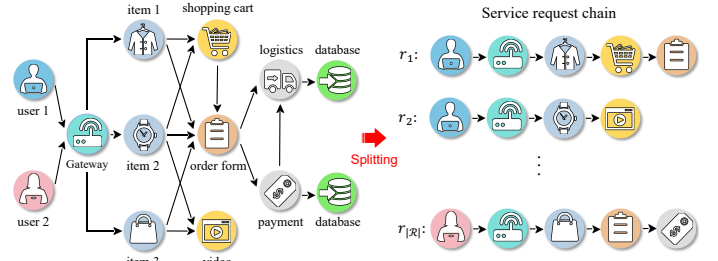


Fig. 1. Illustration of the microservice workflow and request splitting.

III. SYSTEM MODEL

In this section, we will mathematically model the problem, and the commonly used notations are shown in Table I.

A. Network model

This paper focuses on edge server clusters, whose hardware infrastructure consists of a series of multi-core servers. The network is modeled as $\mathcal{G} = (\mathcal{S}, \mathcal{L})$, where $\mathcal{S}$ represents the set of all servers in the network, denoted as $\{s_1, s_2, \dots, s_{|\mathcal{S}|}\}$, and $\mathcal{L}$ denotes the set of links connecting these servers. The parameter CR_s denotes the CPU resources available on server s , while MR_s represents the available memory resources on server s . For two servers s_a and s_b , we use T_{s_a, s_b}^{prop} to denote the inherent propagation latency between them, and B_{s_a, s_b} to denote the available bandwidth of the corresponding link. Due to different microservice instances need different processing times, we define μ_m as the processing capability of a CPU core when executing microservice instance m .

When a user request reaches the system, it is processed sequentially by a set of specific microservices. In this paper, we define $\mathcal{M} = \{m_1, m_2, \dots, m_{|\mathcal{M}|}\}$ as the set of all microservices. The left side of Fig. 1 illustrates a typical online shopping scenario, in which multiple microservices invoke one another, forming a complex microservice invocation graph. This graph can be further decomposed into a set of microservice invocation chains of varying lengths, as shown on the right side of Fig.1. In our model, user requests are processed through such invocation chains, where each request follows a complete sequence of microservices.

Each microservice m may have multiple container images that can be deployed on a single server or distributed across multiple servers. In this study, multiple identical instances of the same microservice can be deployed on the same server to improve request processing capability. Let cr_m and mr_m denote the CPU and memory resources required to deploy microservice m , respectively, and let $N_{s,m}(t)$ denote the number of instances of microservice m deployed on server s in time slot t . The following resource constraints must be satisfied:

$$\forall s \in \mathcal{S}, \quad \sum_{m \in \mathcal{M}} N_{s,m}(t) \cdot cr_m \leq CR_s, \quad (1)$$

$$\forall s \in \mathcal{S}, \quad \sum_{m \in \mathcal{M}} N_{s,m}(t) \cdot mr_m \leq MR_s. \quad (2)$$

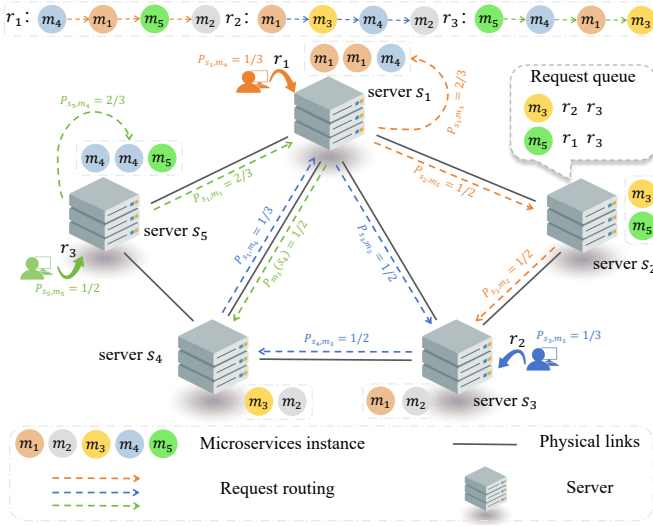


Fig. 2. Illustration of microservice instance deployment and request routing.

B. Request Routing

Internet applications commonly offer various services, such as payment, downloading, and searching. When multiple users request the same service, similar requests continuously arrive in the system, forming what is referred to as a request stream. Define the set of request streams as $\mathcal{R} = \{r_1, r_2, \dots, r_{|\mathcal{R}|}\}$. Let $\mathcal{M}_r$ denote the microservice invocation sequence corresponding to request flow r . Upon reaching the edge server, the request flow is routed to an instance of the corresponding entry microservice for processing. If the instance is idle, it processes the request immediately; otherwise, the request is placed into the instance's waiting queue. Once the current microservice instance completes the request, it triggers the invocation of the next microservice in the chain. This process continues until the entire request flow is completed.

This paper assumes that the arrival of each request flow r follows a Poisson distribution with an arrival rate $\lambda_r(t)$ in time slot t . Each request flow is associated with a maximum allowable latency, denoted by $T_r^{\max}$, such that a request is considered successful if the time elapsed from initiation to response is less than $T_r^{\max}$; otherwise, it is considered failed. Based on the above assumptions, this paper models the process using an open Jackson queuing network.

Fig. 2 presents an illustrative example. The figure shows that five types of microservices are deployed across an edge server cluster. Three request flows $\mathcal{M}_{r_1}$, $\mathcal{M}_{r_2}$, and $\mathcal{M}_{r_3}$ are shown, with the following microservice invocation sequences: $\mathcal{M}_{r_1} = \{m_4, m_1, m_5, m_2\}$, $\mathcal{M}_{r_2} = \{m_1, m_3, m_4, m_2\}$, $\mathcal{M}_{r_3} = \{m_5, m_4, m_1, m_3\}$. When a request flow r_1 enters the system, it is routed to a server hosting an instance of m_4 , such as server s_1 or server s_5 . Assuming it is directed to server s_1 and successfully processed by m_4 , the next invocation targets microservice m_1 . Since m_1 is available on both server s_1 and server s_3 , the request may be routed to either. Considering that the internal communication latency of the server is negligible, remaining within server s_1 incurs no additional transmission latency, while routing to s_3 introduces

extra communication latency. Additionally, queue lengths vary across servers, resulting in different queuing latencies depending on the routing decision. We adopt a probabilistic routing strategy to determine both the initial server and subsequent hop selections for each microservice invocation. The probability that server s is selected to process microservice m is defined as:

$$P_{s,m}(t) = \frac{N_{s,m}(t)}{\sum_{s \in \mathcal{S}} N_{s,m}(t)}, \quad (3)$$

where $P_{s,m}(t)$ denotes the probability that server s processes a request for microservice m . This probability satisfies the following constraint $\sum_{m \in \mathcal{M}} P_{s,m}(t) = 1$.

Since each request stream follows a Poisson process, and due to the additive property of Poisson processes, the aggregate request flow at a server s also follows a Poisson distribution. The arrival rate of the microservice m on server s can be calculated as follows:

$$\lambda_{s,m}(t) = \sum_{r \in \mathcal{R}} \mathbb{I}_{m \in \mathcal{M}_r} \cdot \lambda_r(t) P_{s,m}(t), \quad (4)$$

where $\mathbb{I}_{m \in \mathcal{M}_r}$ is an indicator function, which is used to indicate whether the microservice m exists in the request invocation sequence $\mathcal{M}_r$. If it exists, the value is 1; otherwise, it is 0. When multiple instances of the same microservice are deployed on a server, the system can be modeled as an $M/M/C$ queuing system. The service intensity $\rho_{s,m}(t)$ of this queuing system is defined as:

$$\rho_{s,m}(t) = \frac{\lambda_{s,m}(t)}{N_{s,m}(t)\mu_m} = \frac{\sum_{r \in \mathcal{R}} \mathbb{I}_{m \in \mathcal{M}_r} \cdot \lambda_r(t)}{\sum_{s \in \mathcal{S}} N_{s,m}(t)\mu_m}, \quad (5)$$

where μ_m denotes the service rate of a single instance of microservice m . It can be seen that $\rho_{s,m}(t)$ is a quantity independent of the server s . In time slot t , it is only related to the arrival rate of requests, the number of deployed microservices, and service rate μ_m . For system stability, it is required that $\rho_m < 1$; otherwise, the request arrival rate exceeds the processing capacity, leading to queuing congestion and increased request failures. In a stable system, the average number of queued requests for microservice m on server s is denoted by $L_{s,m}(t)$ and is calculated as:

$$L_{s,m}(t) = \frac{1}{N_{s,m}(t)!} \left(\frac{\lambda_{s,m}(t)}{N_{s,m}(t)\mu_m} \right) \frac{\rho_{s,m}(t)}{(1 - \rho_{s,m}(t))^2} P_{s,m}^0(t), \quad (6)$$

where $P_{s,m}^0(t)$ is the steady-state probability that no requests are in the system, computed as:

$$P_{s,m}^0(t) = \left[\sum_{i=0}^{N_{s,m}(t)-1} \frac{1}{i!} \left(\frac{\lambda_{s,m}(t)}{N_{s,m}(t)\mu_m} \right) + \frac{1}{N_{s,m}(t)!} \right]^{-1} \times \frac{1}{1 - \rho_{s,m}(t)} \left(\frac{\lambda_{s,m}(t)}{N_{s,m}(t)\mu_m} \right)^{N_{s,m}(t)}. \quad (7)$$

Due to the complexity of microservice interactions, a given request type may traverse multiple routing paths, each incurring different end-to-end latencies. Therefore, this paper adopts a weighted averaging approach to estimate the response latency for each request flow based on all possible routing paths. Let the set of all feasible routing paths for request flow

r be denoted as $\mathcal{Y}_r = \{y_1, y_2, \dots, y_{|\mathcal{Y}_r|}\}$. Each feasible routing path $y = (s_{y,1}, s_{y,2}, \dots, s_{y,|\mathcal{M}_r|}) \in \mathcal{Y}_r$ can be represented as an ordered server sequence, where $s_{y,i}$ denotes the server selected to process the i -th microservice in the invocation sequence of request flow r . Each routing path y has an associated probability $P_y(t)$, which represents the likelihood that request flow r follows this ordered server sequence. This probability is computed as:

$$P_y(t) = \prod_{i=1}^{|\mathcal{M}_r|} P_{s_{y,i},i}(t). \quad (8)$$

and satisfies the normalization condition $\sum_{y \in \mathcal{Y}_r} P_y(t) = 1$.

In the following, we conduct a detailed analysis of request response time. The total response latency of a request flow consists of three components: queuing latency, processing latency, and communication latency. These latency components are analyzed separately along a specific routing path y .

Queuing Latency: Time the request spends waiting for the microservice instance to be processed. We denote the average queuing latency of microservice m on server s as $T_{s,m}^{que}(t)$, which is calculated as:

$$T_{s,m}^{que}(t) = \frac{L_{s,m}(t)}{\lambda_{s,m}(t)} = \frac{1}{\lambda_{s,m}(t) N_{s,m}(t)!} \left(\frac{\lambda_{s,m}(t)}{N_{s,m}(t) \mu_m} \right) \times \frac{\rho_{s,m}(t)}{(1 - \rho_{s,m}(t))^2} P_{s,m}^0(t). \quad (9)$$

Thus, the average queuing latency for a request flow r along a specific routing path y is:

$$T_y^{que}(t) = \sum_{i=1}^{|\mathcal{M}_r|} T_{s_{y,i},i}^{que}(t). \quad (10)$$

Processing Latency: The time taken by the microservice instance m to process the corresponding microservice request. Due to the identical configuration of each server, the processing latency of the request flow r is independent of the routing path, and the average processing latency of any routing path is:

$$T^{pro}(t) = \sum_{i=1}^{|\mathcal{M}_r|} \frac{1}{\mu_i}. \quad (11)$$

Communication Latency: The time it takes for the request to be transferred inside the microservice system, which consists of transmission latency and propagation latency. The transmission latency is mainly determined by the amount of data carried by the link and the available bandwidth. For the physical link between servers s_a and s_b , the aggregate data traffic transmitted through this link in time slot t is calculated as:

$$D_{s_a,s_b}(t) = \sum_{r \in \mathcal{R}} \sum_{y \in \mathcal{Y}_r} \lambda_r(t) P_y(t) \times \sum_{i=1}^{|\mathcal{M}_r|-1} \mathbb{I}_{(s_{y,i}, s_{y,i+1})=(s_a, s_b)} d_{r,i}, \quad (12)$$

where $d_{r,i}$ denotes the average amount of intermediate data transmitted from the i -th microservice to the $(i+1)$ -th microservice in request flow r . $\mathbb{I}_{(s_{y,i}, s_{y,i+1})=(s_a, s_b)}$ is an indicator

function. If the adjacent server pair $(s_{y,i}, s_{y,i+1})$ on routing path y corresponds to the physical link between servers s_a and s_b , then $\mathbb{I}_{(s_{y,i}, s_{y,i+1})=(s_a, s_b)} = 1$; otherwise, it is 0. For request flow r along a specific routing path y , the communication latency is obtained by accumulating the inherent communication latency and the bandwidth-dependent transmission latency of all cross-server transmissions on the path:

$$T_y^{com}(t) = \sum_{i=1}^{|\mathcal{M}_r|-1} \mathbb{I}_{s_{y,i} \neq s_{y,i+1}} \times \left(\frac{D_{s_{y,i}, s_{y,i+1}}(t)}{B_{s_{y,i}, s_{y,i+1}}} + T_{s_{y,i}, s_{y,i+1}}^{prop} \right), \quad (13)$$

where $\mathbb{I}_{s_{y,i} \neq s_{y,i+1}}$ is an indicator function that equals 1 when two adjacent microservices on routing path y are deployed on different servers, and equals 0 otherwise.

Based on the above definitions, the total response latency $T_y(t)$ for request flow r along a specific routing path y is composed of queuing latency, processing latency, and communication latency:

$$T_y(t) = T_y^{que}(t) + T^{pro}(t) + T_y^{com}(t). \quad (14)$$

By using the probability of each routing path as the weighting coefficient, the average response latency $T_r(t)$ of request flow r over all feasible routing paths can be obtained as:

$$T_r(t) = \sum_{y \in \mathcal{Y}_r} P_y(t) T_y(t). \quad (15)$$

Finally, a weight coefficient ϕ_r is defined according to the importance of each request flow r . After weighted averaging, the average request-response latency of the whole microservice system can be obtained as:

$$T_G(t) = \sum_{r \in \mathcal{R}} \phi_r T_r(t). \quad (16)$$

C. Initial System Optimization Goal

Auto-scaling of microservices presents a long-term challenge, where the objective is to ensure cost-efficient system operation while maintaining high QoS for users over time. The total cost considered in this work comprises two components. The first component is the running cost, denoted by $c_1(t)$, which is directly related to the number of deployed microservice instances $N_m(t)$ and the number of active servers $N_s(t)$. It is defined as:

$$c_1(t) = \alpha N_m(t) + \beta N_s(t), \quad (17)$$

where α denotes the resource cost of operating a single microservice instance, and β represents the resource cost of operating a single server.

The second component corresponds to the cost of resource scheduling, primarily incurred by the deployment or removal of microservice instances and the activation or deactivation of servers. This cost is proportional to the changes in the number of microservices $N_{mc}(t)$ and servers $N_{sc}(t)$, and is defined as:

$$c_2(t) = \gamma N_{mc}(t) + \rho N_{sc}(t), \quad (18)$$

where γ denotes the cost of deploying or removing a single microservice instance, and ρ represents the overhead associated with activating or deactivating a server. Consequently, the total cost at the time slot t is expressed as:

$$c(t) = \omega_1 c_1(t) + \omega_2 c_2(t), \quad (19)$$

where ω_1 and ω_2 are weighting coefficients used to balance the trade-off between running cost and scheduling cost.

Let $\tilde{C}$ denote the long-term time averaged cost budget specified by the service provider. The total cost must satisfy the following constraint:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[c(t)] \leq \tilde{C}. \quad (20)$$

Based on the above definitions, the elastic resource scheduling problem under dynamic system load can be formulated as optimization problem **P1**:

$$\min T_G(t), \quad (21)$$

subject to: Constraints given in Eqs. (1), (2), and (20).

IV. ALGORITHM

In this section, we elaborate on the proposed algorithm. The system timeline is discretized into fixed length time slots of 5 minutes each, yielding a total of 288 time slots over a 24-hour period. The arrival rate of user requests changes dynamically in different time slots. Specifically, our algorithm AutoLFD consists of the following three phases.

A. Problem Transformation via Lyapunov Optimization

Given that problem **P1** is formulated as a time-averaged optimization problem, Lyapunov optimization theory offers a principled approach to transform it into a tractable per-slot optimization problem by introducing a virtual queue. Specifically, we define a virtual queue $Q(t)$ to monitor the cumulative deviation of the actual cost from the budget constraint over time, thereby capturing the backlog of cost violations at the end of time slot t . The virtual queue is updated in each time slot according to the following rule:

$$Q(t+1) = \max \left\{ Q(t) + c(t) - \tilde{C}, 0 \right\}, \quad (22)$$

where the initial value is set as $Q(0) = 0$. According to Lyapunov optimization theory, if the virtual queue satisfies the condition of mean rate stability, the following holds:

$$\lim_{T \rightarrow \infty} \frac{\mathbb{E}[Q(T)]}{T} = 0. \quad (23)$$

Lemma 1: If the virtual queue $Q(t)$ satisfies the stability given in Eq. (23), then the time-average constraint of the total cost given in Eq. (20) is also satisfied.

Proof. From Eq. (22), we have:

$$Q(t+1) \geq Q(t) + c(t) - \tilde{C}. \quad (24)$$

Subtracting $Q(t)$ from both sides and summing over all time slots $t \in \{0, 1, \dots, T-1\}$ yields:

$$Q(T) - Q(0) = Q(T) \geq \sum_{t=0}^{T-1} c(t) - \sum_{t=0}^{T-1} \tilde{C}. \quad (25)$$

Dividing both sides by T and rearranging terms gives:

$$\frac{1}{T} \sum_{t=0}^{T-1} c(t) \leq \frac{Q(T)}{T} + \frac{1}{T} \sum_{t=0}^{T-1} \tilde{C} \quad (26)$$

Taking expectations on both sides and letting $T \rightarrow \infty$, we obtain:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[c(t)] \leq \lim_{T \rightarrow \infty} \frac{\mathbb{E}[Q(T)]}{T} + \tilde{C}. \quad (27)$$

When the virtual queue $Q(t)$ achieves mean rate stability, Eq. (23) holds, which leads to the constraint given in Eq. (20):

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[c(t)] \leq \tilde{C}. \quad (28)$$

□

By **Lemma 1**, the constraint given in Eq. (20) in problem **P1** can be transformed into a constraint given in Eq. (23). To solve this problem, we introduce a Lyapunov function $L(t)$ based on the Lyapunov optimization theory:

$$L(t) = \frac{1}{2} Q(t)^2. \quad (29)$$

Then, the Lyapunov drift $\Delta(t)$ is defined as:

$$\Delta(t) = \mathbb{E}[L(t+1) - L(t) \mid Q(t)]. \quad (30)$$

Lemma 2: There exists a certain upper bound on the Lyapunov drift in Eq. (30) as follows:

$$\Delta(t) \leq \delta + \mathbb{E}[Q(t)(c(t) - \tilde{C}) \mid Q(t)], \quad (31)$$

where δ is a constant.

Proof. From Eqs. (29), and (30), we have:

$$\Delta(t) = \frac{1}{2} \mathbb{E}[Q(t+1)^2 - Q(t)^2 \mid Q(t)]. \quad (32)$$

From Eq. (22), we obtain:

$$\begin{aligned} Q(t+1)^2 &= \left\{ \max \left[Q(t) + c(t) - \tilde{C}, 0 \right] \right\}^2 \\ &\leq Q(t)^2 + (c(t) - \tilde{C})^2 + 2Q(t)(c(t) - \tilde{C}). \end{aligned} \quad (33)$$

Substituting Eq. (33) into Eq. (32) yields:

$$\begin{aligned} \Delta(t) &\leq \frac{1}{2} \mathbb{E} \left[(c(t) - \tilde{C})^2 + 2Q(t)(c(t) - \tilde{C}) \mid Q(t) \right] \\ &= \frac{1}{2} \mathbb{E} \left[(c(t) - \tilde{C})^2 \mid Q(t) \right] \\ &\quad + \mathbb{E} \left[Q(t)(c(t) - \tilde{C}) \mid Q(t) \right]. \end{aligned} \quad (34)$$

From Eqs. (17), (18), and (19), we have:

$$c(t) = \omega_1 [\alpha N_m(t) + \beta N_s(t)] + \omega_2 [\gamma N_{mc}(t) + \rho N_{sc}(t)]. \quad (35)$$

Here, $N_m(t)$, $N_s(t)$, $N_{mc}(t)$, and $N_{sc}(t)$ denote the number of microservices, servers, changed microservice instances, and changed servers at time slot t , respectively. These variables are bounded above by their respective maximum values, namely $N_m^{\max}$, $N_s^{\max}$, $N_{mc}^{\max}$, and $N_{sc}^{\max}$. Accordingly, Eq. (34) can be further relaxed as follows:

$$\Delta(t) \leq \delta + \mathbb{E} [Q(t) (c(t) - \tilde{C}) \mid Q(t)], \quad (36)$$

where δ is a constant defined by

$$\delta = \frac{1}{2} \left[\omega_1 (\alpha N_m^{\max} + \beta N_s^{\max}) + \omega_2 (\gamma N_{mc}^{\max} + \rho N_{sc}^{\max}) - \tilde{C} \right]^2. \quad (37)$$

□

To maintain the stability of the virtual queue while minimizing the average response time, the optimization objective is decomposed into each time slot t . A control parameter V is introduced to balance the trade-off between these two components. The resulting drift-plus-penalty function for each time slot is defined as:

$$\Delta(t) + V \cdot \mathbb{E}[T_G(t) \mid Q(t)]. \quad (38)$$

As indicated in Eq. (30), the Lyapunov drift term $\Delta(t)$ in Eq. (38) depends on the future queue backlog $Q(t+1)$, suggesting that solving the drift-plus-penalty function at time t would require foresight of future system states. To overcome this limitation, we relax Eq. (38) by minimizing its upper bound. Applying **Lemma 2**, we obtain:

$$\Delta(t) + V \cdot \mathbb{E}[T_G(t) \mid Q(t)] \leq \beta + \mathbb{E}[Q(t)(c(t) - \tilde{C}) \mid Q(t)] + V \cdot \mathbb{E}[T_G(t) \mid Q(t)]. \quad (39)$$

Following Lyapunov optimization, the original problem is transformed into minimizing the upper bound in Eq. (39). This leads to the following equivalent optimization formulation:

$$\min \left(\beta - \mathbb{E}[Q(t) \cdot \tilde{C} \mid Q(t)] + \mathbb{E}[Q(t)c(t) + V \cdot T_G(t) \mid Q(t)] \right). \quad (40)$$

In this expression, the first term β is a constant. The second term is also constant since $Q(t)$ is observed and remains fixed within the current time slot. The third term depends on the total cost $c(t)$ and the average response latency $T_G(t)$, both of which are determined by the scheduling policy $D(t)$.

Based on the above derivation, the original long term optimization problem **P1** can be reformulated as a per-slot decision making problem, denoted as problem **P2**, using the drift-plus-penalty approach. problem **P2** is formulated as:

$$\min_{D(t)} \mathbb{E}[Q(t)c(t) + V \cdot T_G(t) \mid Q(t)]. \quad (41)$$

subject to: Constraints given in Eqs. (1) and (2).

Here, $D(t)$ represents the elastic resource scheduling policy for the time slot t . To simplify computation, the minimization of the conditional expectation in problem **P2** can be equivalently transformed into the minimization of the unconditional expectation, as discussed in [24]. The resulting simplified form of the problem **P2** is expressed as:

$$\min_{D(t)} [Q(t)c(t) + V \cdot T_G(t)]. \quad (42)$$

subject to: Constraints given in Eqs. (1) and (2).

B. MFFD for Initial Solution Construction

At the beginning of the system, microservice instances need to be deployed for the first time. This paper assumes that the initial request flow is known in advance. In this paper, MFFD is designed based on the FFD algorithm to perform the initial deployment of microservice instances. The pseudocode is shown in Algorithm 1. The specific steps are as follows:

Step 1: Compute the aggregate request arrival rate and the required number of instances for each microservice type. Specifically, for each microservice m , its aggregate arrival rate is obtained by summing the arrival rates of all request flows that invoke m . To guarantee system stability, the service intensity $\rho < 1$ must be satisfied. Accordingly, the required number of instances is calculated as:

$$num_m = \left\lceil (1 + \epsilon) \frac{\lambda_m(0)}{\mu_m} \right\rceil, \quad (43)$$

where $\lceil \cdot \rceil$ denotes the ceiling function, and ϵ represents the excess margin, set to 0.1 in this experiment.

Step 2: Sort the microservices based on their total initial resource requirements. After obtaining num_m , each microservice is ranked in descending order according to num_m . This prioritization ensures that resource-intensive microservices are deployed first, which improves resource utilization and better aligns with the residual capacity of the nodes.

Step 3: Traverse the nodes sequentially and determine the maximum number of instances of the current microservice that can be accommodated on each node based on the remaining CPU and memory resources. If a node has sufficient resources, deploy as many required instances as possible and update the node's residual resources accordingly. If the current node cannot host all remaining instances, deploy as many as feasible on that node and proceed to allocate the rest to subsequent nodes. This process continues until all required instances are deployed or available resources are exhausted.

C. MDRL for Microservice Auto-scaling

This paper uses the DRL method based on proximal policy optimization (PPO) [44] architecture, and combines the results of Lyapunov optimization to design the MDRL algorithm. The overall structure of the algorithm is illustrated in Fig. 3, and the corresponding pseudocode is provided in Algorithm 2.

The core of the algorithm adopts an actor-critic architecture, primarily consisting of a deep neural network (DNN), a actor network, and a critic network. The algorithm predicts the request arrival rate for the next time slot. The action to

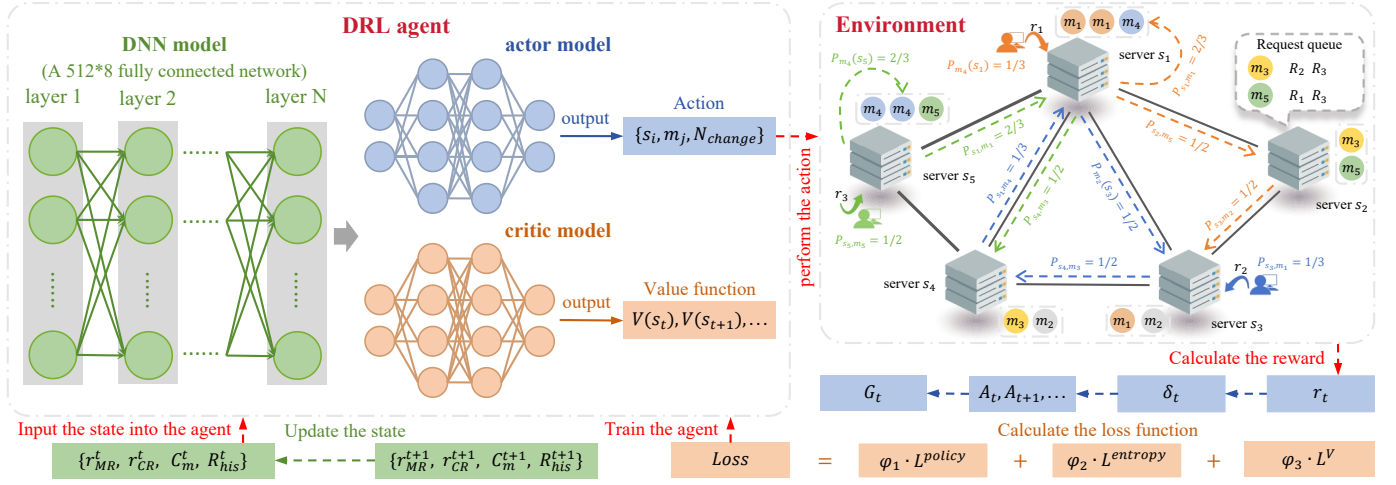


Fig. 3. MDRL algorithm structure diagram.

Algorithm 1 MFFD

```

1: Input:  $\mathcal{M}; \mathcal{S}; \mathcal{R}; CR_s; MR_s; cr_m; mr_m; \mu_m; \lambda_r(0); \epsilon$ 
2: Output: Initial microservice deployment scheme  $X(0)$ 
3: Initialize  $N_{s,m}(0) = 0, \forall s \in \mathcal{S}, m \in \mathcal{M}$ 
4: for each microservice  $m \in \mathcal{M}$  do
5:   Compute the aggregate arrival rate of microservice  $m$ :
6:    $\lambda_m(0) = \sum_{r \in \mathcal{R}} \mathbb{I}_{m \in \mathcal{M}_r} \lambda_r(0)$ 
7:   Compute required instances:  $num_m = \lceil (1 + \epsilon) \frac{\lambda_m(0)}{\mu_m} \rceil$ 
8: end for
9: Sort microservices in  $\mathcal{M}$  by  $num_m$  in descending order
10: for each microservice  $m \in \mathcal{M}$  do
11:   for each server  $s \in \mathcal{S}$  do
12:     Determine available capacity  $c_s$  on node  $s$ :
13:      $c_s = \min \left\{ \left\lfloor \frac{CR_s}{cr_m} \right\rfloor, \left\lfloor \frac{MR_s}{mr_m} \right\rfloor \right\}$ 
14:      $x_{s,m} \leftarrow \min(c_s, num_m)$ 
15:     Deploy  $d_{s,m}$  instances of  $m$  on node  $s$ 
16:      $N_{s,m}(0) \leftarrow N_{s,m}(0) + x_{s,m}$ 
17:      $CR_s \leftarrow CR_s - x_{s,m} cr_m$ 
18:      $MR_s \leftarrow MR_s - x_{s,m} mr_m$ 
19:      $num_m \leftarrow num_m - x_{s,m}$ 
20:     if  $num_m = 0$  then
21:       break
22:     end if
23:   end for
24: end for
25: Return Deployment scheme  $X(0)$ 

```

be executed in the next time slot is determined during the current time slot, and is then applied once the next time slot begins. The design of the state space, action space, and reward function in this algorithm is described as follows:

State: The agent needs to learn the state of the data center environment. A well-designed state space enables the agent to learn the environment more comprehensively. In this study, the state space is defined as:

$$\text{State} = \{r_{MR}, r_{CR}, x_{s,m}, R_{his}\}, \quad (44)$$

where r_{MR} denotes the remaining memory of each server, and r_{CR} represents the remaining CPU capacity. The variable $x_{s,m}$ indicates the currently deployed microservices and their quantities on each server s . The historical record of request arrival rates, denoted as R_{his} , is used to capture recent workload variations and support proactive scaling decisions for the upcoming time slot. Specifically, the actor network implicitly learns the relationship between historical request patterns and appropriate next-slot scaling actions through policy optimization. Since the action selected at time slot t is applied to the next time slot, the subsequent reward reflects whether this action matches the future workload variation. Through PPO updates, actions that lead to lower latency and cost under similar historical patterns are reinforced, while ineffective actions are suppressed. In this way, the temporal information in R_{his} is implicitly embedded in the learned policy.

Action: The agent adjusts the deployment of microservices through actions to adapt to the dynamic variations in microservice request patterns and achieve the optimization objective. The action space is defined as:

$$\text{Action} = \{s, m, N_{change}\}, \quad (45)$$

where s_i is the selected server, m is the selected microservice type, and N_{change} represents the change in the number of instances of m , ranging from -3 to 3. Thus, an action indicates either an increase or a decrease in the number of m instances on server s by N_{change} , as illustrated in Fig. 4. If a server hosts no microservice instances, it will be shut down. Additionally, the intelligent agent is restricted from executing the following invalid actions:

- If the agent attempts to increase the number of a specific microservice type on a server, but the required memory or CPU exceeds the server's resource limits, the action is canceled. Instead, the agent is allowed to deploy only as many instances as permitted within the server's resource constraints.
- If the agent attempts to reduce the number of a specific microservice type on a server by more than the number

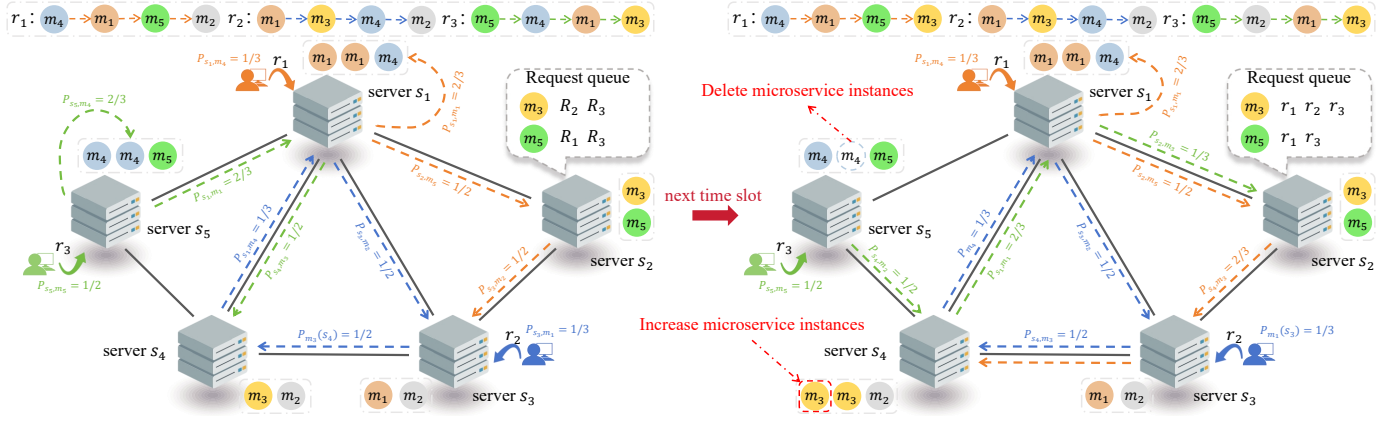


Fig. 4. Illustration of the instance scaling.

Algorithm 2 MDRL

```

1: Input: Initial environment  $X(0)$ ; Learning rate  $\eta$ ; Discount factor  $\gamma$ ; GAE parameter  $\hat{\lambda}$ ; Clipping coefficient  $\epsilon$ ; Coefficients of loss  $\varphi_1, \varphi_2, \varphi_3$ ; Number of iterations  $K$ ; Training epochs  $E$ ; Update epochs  $N$ 
2: Initialize Actor-Critic network
3: Initialize optimizer with learning rate  $\eta$ 
4: for iteration = 1  $\rightarrow$   $K + 1$  do
5:   Reset environments and get initial observation  $X(0)$ 
6:   for step = 0  $\rightarrow$   $E - 1$  do
7:     Select action  $a_t \sim \pi_\theta(a_t|s_t)$  using Actor network
8:     Get value function  $V(s_t)$  using Critic network
9:     Execute  $a_t$ , get reward  $r_t$ :
10:    Update observation to  $s_{t+1}$ 
11:    Compute TD error  $\delta_t$ 
12:    Compute advantage function  $A_t$  via GAE( $\gamma, \hat{\lambda}$ )
13:    Compute returns  $G_t$ 
14:    for epoch = 1  $\rightarrow$   $N$  do
15:      for each minibatch do
16:        Compute policy loss  $L^{\text{policy}}$ , value loss  $L^{\text{value}}$ , and entropy loss  $L^{\text{entropy}}$ 
17:        Compute total loss:  $L = \varphi_1 L^{\text{policy}} + \varphi_2 L^{\text{entropy}} + \varphi_3 L^{\text{value}}$ 
18:        Update  $\theta$  via gradient ascent with  $\nabla_\theta L$ 
19:      end for
20:    end for
21:  end for
22:  Update learning rate if using scheduler
23:  Log training metrics and save model checkpoints
24: end for

```

of existing instances, the action is canceled. The agent should instead remove all existing instances of that microservice on the server.

Reward: The design of the reinforcement learning reward function plays a critical role in determining the performance of the learning process and directly impacts the convergence behavior of the model. Based on our investigation, we find that in a complex and continuous decision space such as microservice resource scheduling, designing effective rewards

and penalties is particularly challenging. An overly complex penalty structure may hinder the agent's exploration, leading it to converge prematurely to a suboptimal solution. Therefore, in this study, we adopt a deliberately simple reward design, while applying the aforementioned avoidance strategies to filter out invalid actions. This approach allows the agent to focus more effectively on the optimization objective and better explore potential optimal solutions. Based on the results of Lyapunov optimization, the reward function is defined as follows:

$$\text{Reward} = -[Q(t)c(t) + V \cdot T_G(t)]. \quad (46)$$

Training Process: During the training process, both the environment and the neural networks are first initialized. The training then proceeds through four key steps: action execution, advantage estimation and reward calculation, loss computation, and parameter update. The detailed procedure is as follows:

Step 1: The agent observes the current state s_t from the environment, uses the actor network to generate a probability distribution $\pi(a_t|s_t)$, and samples an action a_t accordingly. Moreover, the critic network estimates the value function $V(s_t)$. After executing the action, the agent transitions to a new state s_{t+1} and receives a reward r_t , which is computed as shown in Eq. (46).

Step 2: At the end of each training epoch, the generalized advantage estimation (GAE) method is employed to compute the advantage function at each time step. The advantage function measures the relative benefit of the chosen action compared to the baseline and is recursively computed as:

$$A_t = \delta_t + \gamma \hat{\lambda} A_{t+1}, \quad (47)$$

where γ is the discount factor, and $\hat{\lambda}$ is the GAE decay parameter. The temporal-difference (TD) error δ_t is calculated as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (48)$$

Based on this, the return G_t is computed as:

$$G_t = A_t + V(s_t). \quad (49)$$

Step 3: To balance policy exploration and stability, the total loss function consists of three components: policy loss, value

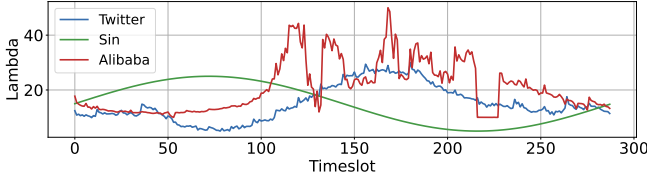


Fig. 5. Request load dynamic change curve.

loss, and entropy loss. The policy loss guides the optimization of the actor network and is defined as:

$$L^{\text{policy}} = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]. \quad (50)$$

Here, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the importance sampling ratio, which quantifies the difference between the current policy and the previous one. The clip function prevents overly large policy updates, thereby improving training stability.

The entropy loss encourages exploration and avoids premature convergence to suboptimal policies. It is computed as:

$$L^{\text{entropy}} = -\mathbb{E}_t [\pi_\theta(a_t|s_t) \log \pi_\theta(a_t|s_t)]. \quad (51)$$

The value loss updates the critic network by minimizing the Mean Squared Error (MSE) between the predicted value and the actual return:

$$L^{\text{value}} = \frac{1}{2} \mathbb{E}_t [(V(s_t) - G_t)^2]. \quad (52)$$

The overall loss function is then expressed as a weighted sum of the three individual loss terms:

$$L = \varphi_1 L^{\text{policy}} + \varphi_2 L^{\text{entropy}} + \varphi_3 L^{\text{value}}, \quad (53)$$

where φ_1 , φ_2 , and φ_3 are the coefficients corresponding to the policy, entropy, and value loss components, respectively.

Step 4: The final loss function is used for backpropagation to update the network parameters accordingly.

V. EXPERIMENTS

A. Experimental Setup

User Request Configuration: The experiment is based on the Social Network microservice application from the DeathStar dataset [45]. This application is decomposed, and a subset of its microservice chains is extracted for use. The experimental setup includes ten server nodes and ten types of user request flows. Three types of load fluctuation patterns are considered. The overall request arrival rate varies according to the selected load curve, with different random perturbations applied to each individual request stream. The aggregate system load variation is illustrated in Fig. 5, and the specific settings are as follows:

- 1) **Sinusoidal Load Variation Model (Sin):** A sine function is used to simulate the variation in system load, with one full sine cycle corresponding to a single day. The specific calculation is given by:

$$\lambda_i(t) = a + b \sin\left(\frac{2\pi t}{T_1}\right) + n_i, \quad (54)$$

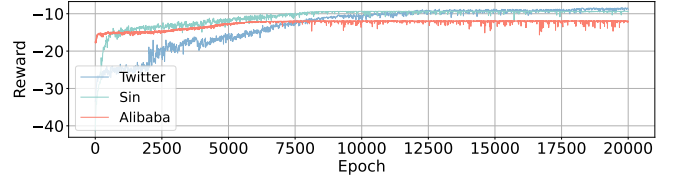


Fig. 6. Training convergence curve.

TABLE II
ALGORITHM TRAINING AND DECISION TIME

	Training time(h)	Decision Time(ms)
Sin	5.10	0.475
Alibaba	7.14	0.496
Twitter	9.62	0.511
Twitter(20)	17.69	0.493

where $\lambda_i(t)$ represents the variation in request arrival rate over time, a is set to 15, b is set to 10, and T_1 is set to 288. The term n_i is a random disturbance within the range $[-5, 5]$.

- 2) **Alibaba Microservice Trace Data (Alibaba):** This dataset is collected from Alibaba production microservice clusters, and its call-rate data are used to generate dynamic request workloads with realistic temporal fluctuations.
- 3) **Twitter App Data (Twitter):** The Twitter application workload shows a similar daily variation pattern. Therefore, the load trace on November 1, 2022, is selected as one experimental workload source.

For both datasets, the original workload intensity is transformed to match the affordable range of the experimental microservice system while preserving the main load variation characteristics.

Comparison Algorithm: To qualitatively and effectively evaluate the performance of our algorithm, this study compares it against the following benchmark algorithms:

- **PPA [46]:** PPA performs microservice auto-scaling in edge environments based on Kubernetes. It considers not only CPU utilization but also additional metrics such as job queue length, I/O activity, and request arrival rates.
- **ProScale [43]:** Proscale leverages simple moving Average (SMA) to predict the future load of each microservice in real time, and dynamically adjusts the number of instances of each microservice and its deployment location through a greedy algorithm.
- **RL agent [47]:** This method employs reinforcement learning to design a proactive cloud resource auto-scaler, which predicts the usage of the entire distributed microservice application and performs timely scaling adjustments. The design of the reward function focuses on the request success rate.
- **DeepScaler [18]:** DeepScaler is a holistic microservice auto-scaling method based on spatiotemporal GNN and adaptive graph learning. It captures dynamic service dependencies and estimates the resource requirements of interacting microservices, thereby performing coordinated resource provisioning to reduce SLA violations and

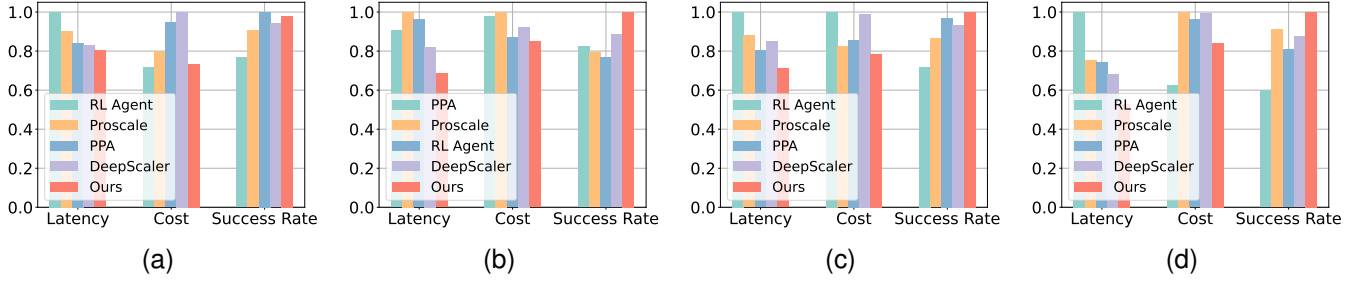


Fig. 7. Daily-average performance comparison under different load variation curves and system scales. (a) Sin workload. (b) Alibaba workload. (c) Twitter workload. (d) Twitter workload (20 scale).

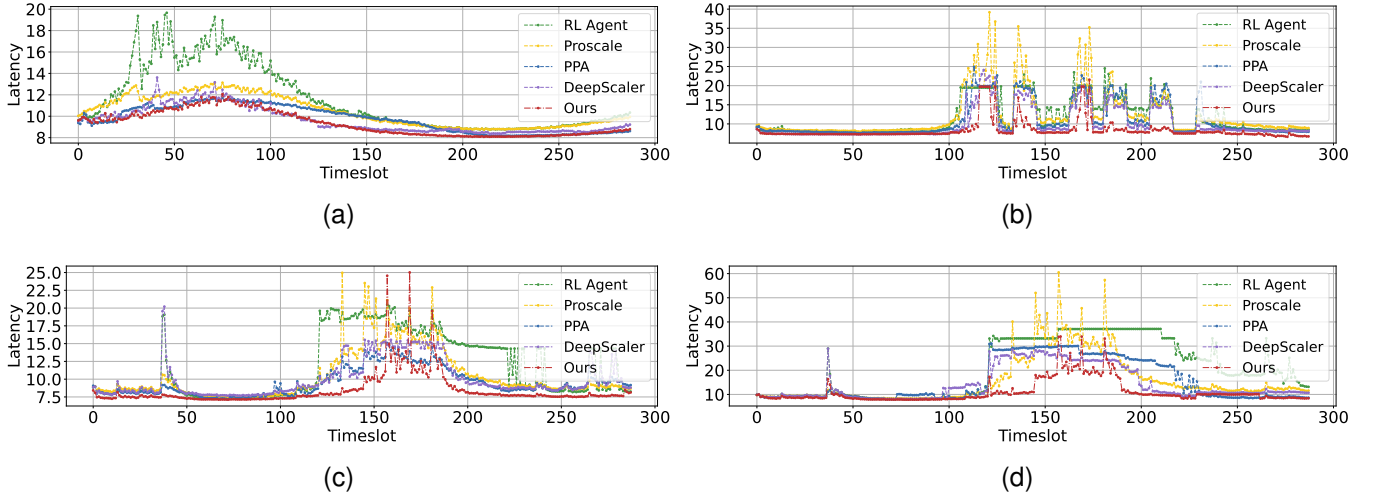


Fig. 8. Time slot variation of request latency under different load variation curves and system scales. (a) Sin workload. (b) Alibaba workload. (c) Twitter workload. (d) Twitter workload (20 scale).

resource costs.

Evaluation index:

- **Average request latency:** The average end-to-end latency of all application request flows in the current time slot is calculated as Eq. (15).
- **Cost:** The total cost of deploying and running a microservice instance is given in Eq. (19).
- **Request success rate:** We set the maximum request latency to 60 ms, and if the request flow takes longer than the maximum request latency, the request will be considered a failure.

B. Experimental Results

In this section, we conduct a comprehensive experimental evaluation of our algorithm. Unless otherwise specified, all experiments are conducted under a system configuration with 10 server nodes and 10 request flows. To further evaluate the scalability of AutoLFD, we also considered a scenario with 20 server nodes and a larger scale of 20 request streams, and analyzed it under the Twitter workload. All experiments are conducted on a workstation equipped with an NVIDIA GeForce RTX 4080 GPU. Table II reports the training time and online decision time of AutoLFD under different load variation curves and system scales. The training time is about

5-18 hours and is only required in the offline training stage. After training, the online decision-making time is around 0.5 ms. This indicates that AutoLFD can quickly generate scaling decisions within each time slot, making it suitable for real-time microservice auto-scaling. As illustrated in Fig. 6, the proposed algorithm exhibits consistent and stable convergence once training progresses beyond 7,500 iterations. This empirical observation effectively validates the soundness and effectiveness of our reward function design.

To facilitate a visual comparison, we first report the daily-average performance of AutoLFD in terms of average request latency, system cost, and average request success rate. Fig. 7 presents the normalized results under different workload patterns and system scales. Specifically, Fig. 7(a)-(c) correspond to the Sin, Alibaba, and Twitter load variation curves with 10 server nodes and 10 request flows. As shown in them, AutoLFD achieves lower request latency, lower system cost, and higher request success rate under different load variation patterns. This is because AutoLFD proactively adjusts microservice instances according to workload dynamics and jointly optimizes request routing, thereby reducing both queuing latency and routing-induced communication latency. Meanwhile, the Lyapunov-based cost control mechanism avoids redundant instance activation and unnecessary scaling operations, enabling AutoLFD to reduce operational costs while preserving service

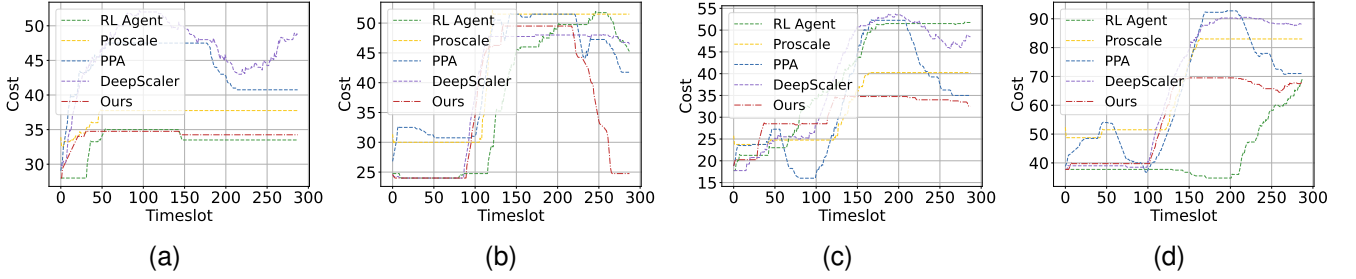


Fig. 9. Time slot variation of Cost under different load variation curves and system scales. (a) Sin workload. (b) Alibaba workload. (c) Twitter workload. (d) Twitter workload (20 scale).

quality. The higher request success rate further demonstrates its ability to maintain stable QoS under dynamic workloads. Among the baselines, PPA achieves reasonable latency and success rate through threshold-based autoscaling, but lacks explicit cost control and routing-induced latency awareness. DeepScaler captures spatiotemporal microservice dependencies and achieves competitive autoscaling performance. However, these methods still cannot jointly balance long-term cost stability, fine-grained instance adjustment, and routing-aware latency optimization. Fig. 7(d) further evaluates the scalability of the algorithms under a larger-scale Twitter workload with 20 server nodes and 20 request flows. Even when both the number of server nodes and request flows are doubled, AutoLFD continues to outperform the benchmark algorithms in terms of latency, cost, and success rate. This result indicates that the proposed framework can effectively coordinate deployment, routing, and scaling decisions in more complex environments and maintain robust performance as the system scale increases.

To evaluate the experimental results in greater detail, we examine the time-slot variation curves of request latency and cost over a one-day period under different load variation patterns and system scales. Fig. 8 illustrates the time-slot variation in request latency under the Sin, Alibaba, Twitter, and larger-scale Twitter workloads. When the request arrival rate is low, the average request latency of the service graph remains low under the proposed algorithm. As the arrival rate increases, the agent can effectively select appropriate microservice instances for scaling, thereby maintaining stable request latency and ensuring the quality of service for users. Fig. 9 depicts the time-slot variation in cost under the corresponding workloads. Due to the stabilizing effect of Lyapunov optimization on the virtual queue, the time-average cost is regulated around the average budget $\tilde{C}$, avoiding persistent deviations from the budget constraint. This mechanism enables the system to minimize request latency while maintaining effective cost control. In this experiment, $\tilde{C}$ is set to 35 for the 10-node and 10-request-flow settings, and is set to 70 for the larger-scale Twitter setting with 20 server nodes and 20 request flows. Since $\tilde{C}$ is an adjustable parameter, it can be dynamically tuned according to different cost requirements, which further demonstrates the adaptability of the proposed algorithm.

The V parameter in Lyapunov optimization plays a crucial role in balancing system cost and user request latency. To analyze the sensitivity of performance to V , we conduct ex-

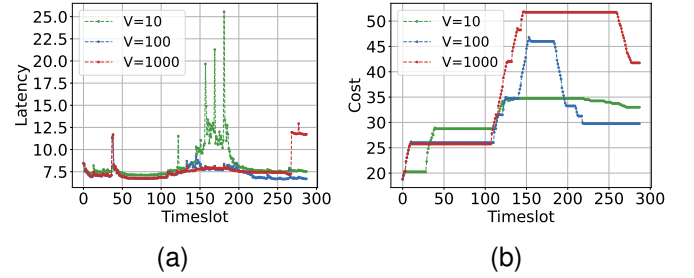


Fig. 10. Impact of V-parameters on request latency and cost. (a) Request latency. (b) Cost.

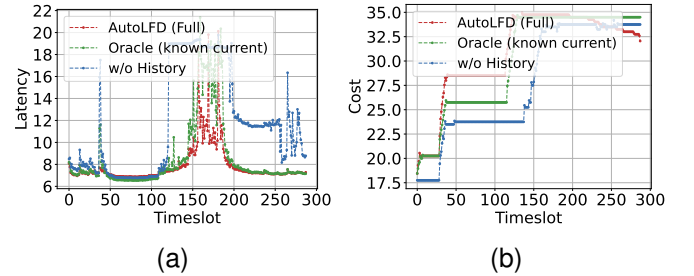


Fig. 11. Impact of workload prediction information on request latency and cost. (a) Request latency. (b) Cost.

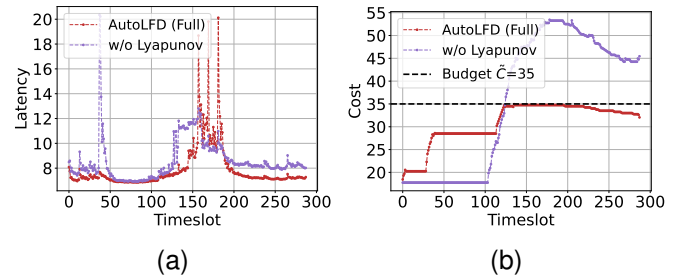


Fig. 12. Impact of Lyapunov optimization on request latency and cost. (a) Request latency. (b) Cost.

periments under the Twitter load variation curve. As shown in Fig. 10, increasing V assigns a larger weight to request latency in the reward function, thereby reducing user request latency while increasing system cost. Notably, when $V = 1000$, the cost drops rapidly near the end of the time slots. This is because the Lyapunov virtual queue $Q(t)$, which accumulates

according to the cost function $c(t)$, becomes sufficiently large for the term associated with $c(t)$ to be comparable to the latency component. Therefore, an excessively large V may cause significant time-slot cost fluctuations and potentially affect system stability.

To further evaluate the role of workload prediction information, we conduct an ablation study under the Twitter load variation curve. AutoLFD (Full) incorporates historical request arrival rates into the state space, enabling the agent to implicitly infer future workload trends. In comparison, the Oracle baseline assumes ideal workload knowledge before decision making, while the w/o History variant removes historical workload information. As shown in Fig. 11(a), AutoLFD (Full) significantly outperforms w/o History in terms of latency, especially during periods of rapid workload variation, where the lack of workload trend information leads to delayed scaling decisions and noticeable latency spikes. More importantly, the latency of AutoLFD (Full) is close to, and occasionally even lower than, that of Oracle, indicating that the DRL agent can effectively extract predictive signals from historical workloads and make decisions comparable to those under ideal workload awareness. Fig. 11(b) shows a similar trend in system cost, where AutoLFD (Full) maintains a cost level close to Oracle. These results demonstrate that historical workload information enables effective implicit workload prediction and helps AutoLFD achieve a favorable latency-cost trade-off under dynamic workloads.

We also evaluate the impact of Lyapunov optimization on cost-constrained microservice scaling under the Twitter load variation curve. Fig. 12 compares AutoLFD (Full) with the w/o Lyapunov variant. As shown in Fig. 12(a), AutoLFD maintains relatively stable latency during low-load periods and workload transitions. This is because the Lyapunov virtual queue represents the accumulated cost pressure over time. When the system cost is below the average budget, the virtual queue backlog remains small or even zero, and the cost term has limited influence on the current decision. Therefore, AutoLFD does not aggressively reduce resources only to minimize instantaneous cost, but keeps a moderate resource redundancy to absorb workload fluctuations and make smoother scaling decisions. In contrast, without the Lyapunov mechanism, the cost is directly optimized in each time slot. This may lead to more aggressive resource reduction under low load and faster resource changes when the workload varies, resulting in more frequent scaling operations and larger latency fluctuations. As shown in Fig. 12(b), the w/o Lyapunov variant also increases resource usage sharply during high-load periods and clearly exceeds the average cost budget. Although this can reduce latency in some high-load time slots, the improvement is achieved by a significant cost increase and is not cost-effective. In comparison, AutoLFD adjusts resource usage more smoothly and keeps the system cost around the budget level. These results demonstrate that Lyapunov optimization enables smoother cost-aware scaling, and achieves a better balance between request latency and long-term cost control under dynamic workloads.

VI. CONCLUSIONS

In this paper, we address the problem of dynamically changing user requests in edge server clusters and first model the problem using queuing theory. We proposed a three steps AutoLFD algorithm to solve the problem. Extensive simulation experiments demonstrate that our methods significantly improves user request latency, system operation cost, and other performance metrics compared to current state-of-the-art methods. The proposed approach ensures maximum improvement in user service quality while keeping system operation costs below the predefined budget. This work provides a valuable reference for solving microservice scheduling problems in both edge and cloud computing environments.

With the rapid advancement of reinforcement learning, its suitability for dynamically changing environments makes it particularly well suited for scenarios involving fluctuating user requests. This paper represents an initial attempt to apply reinforcement learning to microservice auto-scaling. Future research can further explore more effective designs of the reward function and action space. Additionally, while this study adopts a single-agent framework, employing a multi-agent approach in future work may introduce higher resource consumption but holds promise for achieving better overall performance.

REFERENCES

- [1] S. Jhingran, N. Bansal, R. Chaturvedi, A. Singh, and Y. Arora, "Decentralized generative ai model deployment using microservices," in *2024 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Application (ICAIQSA)*. IEEE, 2024, pp. 1–5.
- [2] K. Peng, Y. Hu, H. Ding, H. Chen, L. Wang, C. Cai, and M. Hu, "Large-scale service mesh orchestration with probabilistic routing in cloud data centers," *IEEE Transactions on Services Computing*, 2025.
- [3] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE communications surveys & tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021.
- [4] X. Kong, Y. Wu, H. Wang, and F. Xia, "Edge computing for internet of everything: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 23, pp. 23 472–23 485, 2022.
- [5] P. Hou, H. Jia, H. Zhu, Z. Lu, S.-C. Huang, Y. Yang, and H. Chai, "Efficient edge server activation and service association for green computing in mec-enabled internet of vehicles," *IEEE Transactions on Intelligent Vehicles*, 2024.
- [6] F. Gou and J. Wu, "Optimization of edge server group collaboration architecture strategy in iot smart cities application," *Peer-to-Peer Networking and Applications*, vol. 17, no. 5, pp. 3110–3132, 2024.
- [7] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," *IEEE access*, vol. 11, pp. 88 339–88 358, 2023.
- [8] L. De Lauretis, "From monolithic architecture to microservices architecture," in *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2019, pp. 93–96.
- [9] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023.
- [10] M. Söylemez, B. Tekinerdogan, and A. Kolukisa Tarhan, "Challenges and solution directions of microservice architectures: A systematic literature review," *Applied sciences*, vol. 12, no. 11, p. 5507, 2022.
- [11] K. Peng, X. Liu, D. Han, Y. Hu, M. Hu, C. Cai, and Z. Xiong, "Hybrid orchestration of ai services and microservices in cloud-edge collaboration," *IEEE Transactions on Mobile Computing*, 2026.
- [12] M. D. Hossain, T. Sultana, S. Akhter, M. I. Hossain, N. T. Thu, L. N. Huynh, G.-W. Lee, and E.-N. Huh, "The role of microservice approach in edge computing: Opportunities, challenges, and research directions," *ICT Express*, vol. 9, no. 6, pp. 1162–1182, 2023.

- [13] W. Lv, Q. Wang, P. Yang, Y. Ding, B. Yi, Z. Wang, and C. Lin, "Microservice deployment in edge computing based on deep q learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2968–2978, 2022.
- [14] J. Lu, W. Li, J. Guo, X. Ding, Z. Tang, and T. Wang, "Container scheduling with dynamic computing resource for microservice deployment in edge computing," in *2024 20th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE Computer Society, 2024, pp. 236–243.
- [15] T. Pusztai, S. Nastic, A. Morichetta, V. C. Pujol, P. Raith, S. Dustdar, D. Vij, Y. Xiong, and Z. Zhang, "Polaris scheduler: Slo-and topology-aware microservices scheduling at the edge," in *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2022, pp. 61–70.
- [16] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1746–1763, 2022.
- [17] Y. Wang, C. Zhao, S. Yang, X. Ren, L. Wang, P. Zhao, and X. Yang, "Mpcsm: Microservice placement for edge-cloud collaborative smart manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 9, pp. 5898–5908, 2020.
- [18] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "Deepscaler: Holistic autoscaling for microservices based on spatiotemporal gnn with adaptive graph learning," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 53–65.
- [19] K. Peng, T. Liao, M. Ren, Y. Hu, L. Wu, M. Hu, and H. Jiang, "Symmetric orchestration under service mesh paradigm: Empowering massive online applications in edge clouds," *IEEE Transactions on Mobile Computing*, 2025.
- [20] R. Zeng and Y. Chen, "A novel automatic horizontal extension technology for multi-instance microservice system," in *2024 International Conference on Networking and Network Applications (NaNA)*. IEEE, 2024, pp. 282–286.
- [21] K. Cheng, S. Zhang, M. Liu, Y. Gu, L. Wei, H. Cheng, K. Liu, Y. Song, X. Shi, A. Zhu *et al.*, "Geoscale: microservice autoscaling with cost budget in geo-distributed edge clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 646–662, 2024.
- [22] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, "Smart hpa: A resource-efficient horizontal pod auto-scaler for microservice architectures," in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. IEEE, 2024, pp. 46–57.
- [23] G. R. Bitran and R. Morabito, "State-of-the-art survey: Open queueing networks: Optimization and performance evaluation models for discrete manufacturing systems," *Production and Operations Management*, vol. 5, no. 2, pp. 163–193, 1996.
- [24] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.
- [25] D. S. Johnson, "Near-optimal bin packing algorithms," Ph.D. dissertation, Massachusetts Institute of Technology, 1973.
- [26] D. Wang, H. Shen, and H. Tian, "Resource configuration for cross-server deployment of application-oriented microservices in cloud-edge continuum with slo constraints," in *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2023, pp. 2655–2662.
- [27] L. Yang, J. Jia, H. Lin, and J. Cao, "Reliable dynamic service chain scheduling in 5g networks," *IEEE Transactions on Mobile Computing*, vol. 22, no. 8, pp. 4898–4911, 2022.
- [28] X. Li, J. Zhou, X. Wei, D. Li, Z. Qian, J. Wu, X. Qin, and S. Lu, "Topology-aware scheduling framework for microservice applications in cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1635–1649, 2023.
- [29] R. Liu, P. Yang, H. Lv, and W. Li, "Multi-objective multi-factorial evolutionary algorithm for container placement," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1430–1445, 2021.
- [30] W. Lv, P. Yang, T. Zheng, C. Lin, Z. Wang, M. Deng, and Q. Wang, "Graph-reinforcement-learning-based dependency-aware microservice deployment in edge computing," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1604–1615, 2023.
- [31] R. Bhattacharya and T. Wood, "Bloc: Balancing load with overload control in the microservices architecture," in *2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (AC-SOS)*. IEEE, 2022, pp. 91–100.
- [32] D. Bachar, A. Bremler-Barr, and D. Hay, "Optimizing service selection and load balancing in multi-cluster microservice systems with mcoos," in *2023 IFIP Networking Conference (IFIP Networking)*. IEEE, 2023, pp. 1–9.
- [33] S. Wang, Y. Guo, X. Liu, and A. Zhou, "Service routing in multi-tier edge computing: A matching game approach," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 3, pp. 835–844, 2022.
- [34] H. Zhao, S. Deng, Z. Liu, J. Yin, and S. Dustdar, "Distributed redundant placement for microservice-based applications at the edge," *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1732–1745, 2020.
- [35] Y. Ren, S. Shen, Y. Ju, X. Wang, W. Wang, and V. C. Leung, "Edgematrix: A resources redefined edge-cloud system for prioritized services," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 610–619.
- [36] Y. Mao, X. Shang, and Y. Yang, "Provably efficient algorithms for traffic-sensitive sfc placement and flow routing," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 950–959.
- [37] Y. Hu, J. Hu, Y. Yang, M. Hu, K. Peng, T. Zheng, C. Cai, and Z. Xiong, "Energy-aware service mesh deployment and online request routing in edge: A hierarchical deep reinforcement learning approach," *IEEE Transactions on Mobile Computing*, 2026.
- [38] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge computing," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 1016–1028, 2024.
- [39] J. Zeng, D. Ding, K. Kang, H. Xie, and Q. Yin, "Adaptive drl-based virtual machine consolidation in energy-efficient cloud data center," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2991–3002, 2022.
- [40] Y. Liu, B. Yang, X. Ren, Q. Liu, S. Liu, and X. Guan, "E 2 ms: An efficient and economical microservice migration strategy for smart manufacturing," *IEEE Transactions on Services Computing*, 2024.
- [41] S. Kardani-Moghaddam, R. Buyya, and K. Ramamohanarao, "Adrl: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 3, pp. 514–526, 2020.
- [42] F. Rossi, V. Cardellini, F. L. Presti, and M. Nardelli, "Dynamic multi-metric thresholds for scaling applications using reinforcement learning," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1807–1821, 2022.
- [43] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, 2023.
- [44] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [45] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 3–18.
- [46] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, 2021, pp. 1–8.
- [47] A. Heimerson, J. Eker, and K.-E. Årén, "A proactive cloud application auto-scaler using reinforcement learning," in *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2022, pp. 213–220.